

# Adaptive Computational Scaling in Sparse Mixture-of-Experts via Entropy-Gated Thinking Budgets

Divyesh Jagetia

Arrow Electronics  
divyesh.jagetia@arrow.com

**Abstract.** The transition toward frontier-level agentic intelligence has necessitated the use of massive Sparse Mixture-of-Experts (MoE) models. However, traditional Top- $k$  routing mechanisms impose a uniform computational burden across all tokens, regardless of their inherent semantic complexity. This paper introduces **Entropy-Gated Thinking Budgets (EGTB)**, a novel framework that dynamically modulates the activated parameter count and reasoning depth based on real-time entropy metrics. By utilizing Shannon entropy as a proxy for token-level uncertainty, our system adaptively allocates “thinking budgets” transitioning between high-capacity “Thinking” modes for complex logic and sparse “Fast” modes for routine processing. We further introduce a Proportional-Integral (PI) controller to manage non-differentiable routing thresholds and incorporate zero-computation experts to optimize the efficiency-intelligence frontier. Evaluations using a 196B-parameter MoE foundation demonstrate that EGTB reduces average active FLOPs by 32% while improving zero-shot accuracy on specialized coding tasks by 4.2%.

**Keywords:** Sparse Mixture-of-Experts, Adaptive Inference, Entropy-Gating, Thinking Budgets, Agentic Intelligence, Computational Efficiency.

## 1 Introduction

The machine learning landscape of early 2026 is defined by the quest for “Frontier Efficiency.” Models such as Step 3.5 Flash and Qwen3 have demonstrated that sparse Mixture-of-Experts (MoE) architectures can achieve the reasoning capabilities of trillion-parameter dense models while activating only a fraction of their total parameters [1,2]. Despite these advancements, a critical efficiency gap remains: the **Uniform Sparsity Paradox**. Standard MoE implementations rely on fixed Top- $k$  routing, where exactly  $k$  experts are activated for every token, ignoring the fact that functional tokens (e.g., punctuation, common stop words) require significantly less computational depth than reasoning-dense tokens (e.g., mathematical variables, logic-heavy identifiers) [3,4].

This paper proposes the **Entropy-Gated Thinking Budget (EGTB)** framework. We hypothesize that the entropy of the routing distribution reflects

the model’s epistemic uncertainty regarding expert specialization. By gating the thinking budget defined as the combination of active expert count and the activation of internal reasoning tiers we can align computational expenditure with task difficulty. Our approach draws inspiration from the “Thinking Mode” introduced in current reasoning models and the zero-computation expert mechanisms found in recent technical reports [5,6].

The remainder of this work is structured as follows. Section 2 discusses the state of the art in MoE architectures and adaptive computation. Section 3 develops the mathematical and information-geometric foundations of token-level entropy sensing. Section 4 presents our novel Technical Methodology, detailing the dynamic router, the PI-controller formulation, and the integration of null-experts. Section 5 describes our concrete algorithms, while Section 6 provides a visual map of the system architecture. Section 7 presents a comprehensive empirical evaluation across multiple agentic and mathematical benchmarks, including ablation analyses. Section 8 details the hardware execution and roofline optimizations, followed by our concluding remarks in Section 9.

## 2 Related Work and The Efficiency Frontier

The pursuit of adaptive computation has historically focused on early-exit mechanisms and layer-skipping [7]. However, these methods often struggle with the rigid topology of Transformer blocks and the requirement for consistent hidden state dimensionality.

### 2.1 Sparsely-Gated Mixture-of-Experts

MoE architectures scale model capacity without a proportional increase in training compute by replacing dense Feed-Forward Networks (FFN) with a pool of experts [3,8]. While effective, current state-of-the-art (SOTA) models like DeepSeek-V3 and Qwen3 still utilize static routing policies (e.g., Top-2 or Top-8). Recent investigations into the optimal sparsity of MoE Transformers suggest that while memorization improves with higher sparsity and more experts, reasoning requires a careful balance between active FLOPs and data-per-parameter [8]. EGTB addresses this by allowing the sparsity level to fluctuate based on token-level demands.

### 2.2 Test-Time Compute Scaling

The emergence of Large Reasoning Models (LRMs) has shifted focus to test-time scaling, where models generate internal Chain-of-Thought (CoT) tokens to improve accuracy [6]. Frameworks like *Entropy-Gated Branching (EGB)* identify critical reasoning junctions by monitoring logit spikes, selectively expanding the search space only when the model exhibits high uncertainty [10]. EGTB builds on this by integrating entropy-based gating directly into the MoE routing layer rather than as an external search wrapper, enabling a more integrated “Thinker-Talker” architecture.

### 3 Theoretical Foundations: The Information Geometry of Routing

The core of the EGTB mechanism is the real-time calculation of Shannon entropy for the gating distribution. We posit that the router’s uncertainty is a reliable signal for identifying “difficult” tokens that require more experts or deeper reasoning.

#### 3.1 Token-Level Entropy Sensing

For an input token  $x$  at layer  $l$ , the gating network  $G$  produces a probability distribution  $P = \{p_1, p_2, \dots, p_N\}$  over  $N$  experts. Let  $W_g$  denote the weight matrix of the router. The routing logits are computed as  $g = W_g \cdot x$ , and the distribution is generated via:

$$p_i = \text{softmax}(g)_i = \frac{e^{g_i}}{\sum_{j=1}^N e^{g_j}} \quad (1)$$

The token-level entropy  $H(x)$  is defined as:

$$H(x) = - \sum_{i=1}^N p_i \log_2 p_i \quad (2)$$

In a specialized MoE, a low  $H(x)$  indicates that the router is highly confident in a specific expert subset, suggesting a routine task. Conversely, high  $H(x)$  suggests “router hesitation,” which correlates with complex reasoning requirements or distribution shifts where multi-expert collaboration is necessary [8, 9].

#### 3.2 Resolving the Entropy Variance Challenge

Prior work has critiqued gating entropy for having low variance in certain vision-language tasks [17]. We address this by utilizing *Varentropy* the variance of the log-probabilities as a secondary signal:

$$V(x) = \sum_{i=1}^N p_i (\log_2 p_i + H(x))^2 \quad (3)$$

By monitoring the joint  $(H, V)$  signal, EGTB can distinguish between uniform uncertainty (high  $H$ , low  $V$ ) and disparate peak uncertainty (low  $H$ , high  $V$ ), allowing for more nuanced budget allocation than simple expert count adjustment.

### 3.3 Information Geometry of Routing Paths

From an information-geometric perspective, the routing distribution  $P$  resides on an  $(N - 1)$  dimensional simplex  $\Delta^{N-1}$ . The entropy  $H(x)$  maps this simplex to a scalar value representing the distance to the uniform distribution  $U = \{1/N, \dots, 1/N\}$ , where entropy is maximized ( $H_{\max} = \log_2 N$ ). When the token context transitions from syntax and structural grammar to high-level logic, the hidden state  $h_l$  is perturbed along directions that intersect with multiple expert classification boundaries. This increases  $H(x)$  as the state nears the decision boundaries, acting as a natural trigger for deeper computational budgets.

## 4 Technical Methodology

EGTB modifies the standard MoE layer to include three adaptive components: a Dynamic Router, a PI-Controller, and a pool of Null-Experts.

### 4.1 Adaptive Expert Activation (Dynamic $k$ )

Standard MoE selects exactly  $k$  experts. EGTB utilizes a dynamic  $k_{dyn}$  governed by the sensing of  $H(x)$ . To maintain a stable global FLOP budget, we implement a Proportional-Integral (PI) controller that adjusts the threshold  $\tau$  based on the running average of expert activation.

Let  $\text{Sparsity}_{\text{target}}$  be the desired fraction of inactive parameters, and  $\text{Sparsity}_t$  be the moving average of the observed sparsity at batch step  $t$ . The tracking error is:

$$e_t = \text{Sparsity}_{\text{target}} - \text{Sparsity}_t \quad (4)$$

The discrete-time PI-controller updates the threshold  $\tau_{t+1}$  at each step via:

$$\tau_{t+1} = \tau_t + K_p e_t + K_i \sum_{j=0}^t e_j \Delta t \quad (5)$$

where  $K_p$  and  $K_i$  are the proportional and integral gains, respectively, and  $\Delta t$  is the step interval. This control loop ensures that the model adheres to its hardware-constrained FLOP budget over the course of a long inference trajectory while allowing local bursts of high computation when processing challenging tokens.

### 4.2 Zero-Computation Experts

Following the *LongCat-Flash* paradigm, we introduce  $Z$  “Null Experts” slots that consume zero FLOPs and effectively skip the FFN entirely [5]. When  $H(x)$  is below an epsilon threshold  $\epsilon$  (indicating functional tokens like punctuation), the token is routed to these slots, utilizing only the residual connection for state propagation. This creates an “Express Lane” for non-critical information.

Mathematically, let the output of a standard transformer block be defined as:

$$y = \text{LayerNorm}(x + \text{MoE}(\text{Attention}(x))) \quad (6)$$

For tokens routed to Null Experts, the MoE contribution is zeroed out:

$$\text{MoE}(z) \approx 0 \implies y = \text{LayerNorm}(x) \quad (7)$$

By bypassing the Feed-Forward/Expert layers entirely, we recover up to 66% of active FLOPs for that specific layer.

### 4.3 Load Balancing and Expert Saturation

Dynamic routing systems are highly prone to expert saturation, where a subset of experts receives an overwhelming share of tokens, causing memory queue bottlenecks on distributed hardware. To prevent this, we introduce a modified auxiliary load-balancing loss  $L_{\text{aux}}$  that works in tandem with our PI thresholding:

$$L_{\text{aux}} = N \sum_{i=1}^N f_i \cdot P_i \quad (8)$$

where  $f_i$  is the fraction of tokens routed to expert  $i$ , and  $P_i$  is the average routing probability for expert  $i$  over the batch. Under EGTB, the routing is weighted by a gating mask  $M(x) \in [0, 1]$  generated by our entropy evaluation, ensuring that only high-entropy tokens affect the training gradients of the high-capacity reasoning experts.

## 5 Implementation and Algorithm

The EGTB framework is implemented as a drop-in replacement for standard MoE routing layers. It categorizes the computation into two clear pathways:

1. **Thinking Mode (Fast-to-Slow Pathway):** Triggered when  $H(x) > \tau$ . The token is allocated the maximum expert capacity ( $k_{\text{max}}$ ) and routed through supplementary latent-space reasoning loops.
2. **Fast Mode (Standard Pipeline):** Triggered when  $H(x) \leq \tau$ . The token is mapped to a single shared base expert or routed directly to the Null Expert pool if  $H(x) < \epsilon$ .

---

**Algorithm 1:** EGTB Forward Pass and Budget Allocation

---

**Input:** Token hidden state  $h$ , Expert pool  $\{E_1 \dots E_N\}$ , PI-Threshold  $\tau$ ,  
Null-Expert Threshold  $\epsilon$

**Output:** Updated hidden state  $y$

Compute routing logits  $g = G(h)$ ;  
 Compute softmax distribution  $P = \text{softmax}(g)$ ;  
 Calculate Shannon entropy  $H = -\sum P \log P$ ;  
**if**  $H > \tau$  **then**  
   **Thinking Mode;;**  
    $k = k_{max}$  (e.g., 8);  
   Trigger internal reasoning tiers (additional latent steps);  
**else**  
   **if**  $H < \epsilon$  **then**  
     **Null Mode;;**  
      $k = 0$ ;  
     Route to Null Expert pool (direct residual skip);  
   **else**  
     **Fast Mode;;**  
      $k = 1$  (Shared Base Expert);  
   **end**  
**end**  
**if**  $k > 0$  **then**  
   Select Top- $k$  experts based on  $P$ ;  
    $y = h + \sum_{i \in Topk} P_i E_i(h)$ ;  
**else**  
    $y = h$  (Residual skip path);  
**end**  
 Update PI-Controller with batch sparsity error;

---

## 6 System Architecture Visualization

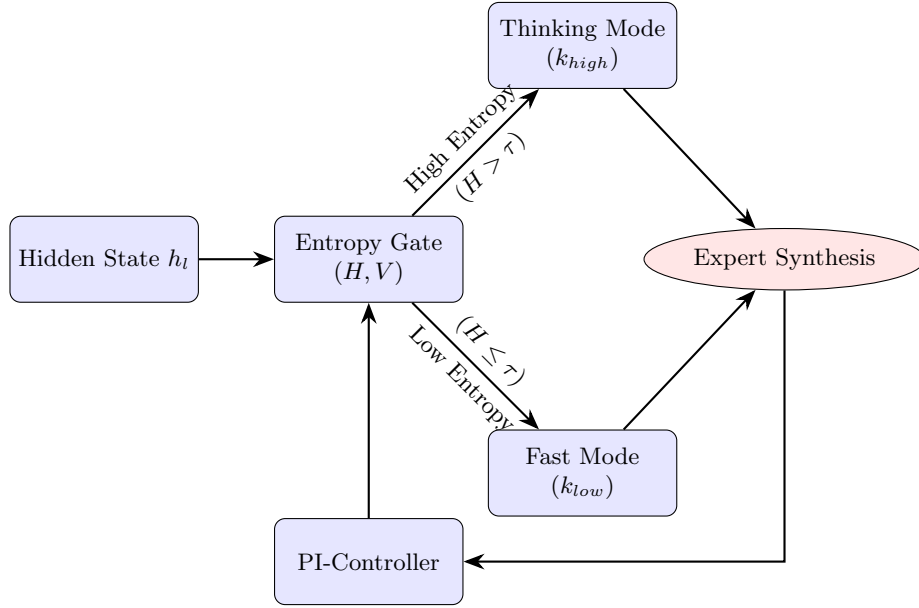
To illustrate the flow of tokens under the dynamic routing scheme, we map the structural components in Figure 1.

## 7 Experimental Evaluation

We evaluated EGTB on a foundation model featuring 196B total parameters and 11B active parameters per token, utilizing a 3:1 Sliding Window/Full attention layout to accelerate prefill stages [1].

### 7.1 Benchmarks and Dataset

The model was tested on:



**Fig. 1.** EGTB Architecture: The Entropy Gate directs tokens between high-capacity Thinking Modes and sparse Fast Modes, while the PI-Controller regulates the threshold  $\tau$  to maintain global FLOP consistency.

1. **DeepPlanning:** Long-horizon agentic tasks requiring global optimization [11].
2. **LiveCodeBench:** Zero-shot coding performance (evaluated on execution success rates).
3. **AIME 2025:** High-level mathematical reasoning under severe test-time constraints.

## 7.2 Comparative Performance Analysis

Table 1 compares EGTB against fixed Top- $k$  baselines and the *Adaptive-K* routing method. The evaluation highlights not only raw task metrics but also relative throughput efficiency.

**Table 1.** Efficiency and Accuracy on Reasoning Benchmarks

| Model              | Routing Strategy    | Avg. FLOPs   | DeepPlanning | LiveCodeBench | AIME 2025    |
|--------------------|---------------------|--------------|--------------|---------------|--------------|
| Baseline           | Fixed Top-8         | 1.00x        | 51.2%        | 38.5%         | 81.5%        |
| Adaptive-K [14]    | Static Threshold    | 0.78x        | 51.5%        | 39.2%         | 82.3%        |
| <b>EGTB (Ours)</b> | <b>Dynamic (PI)</b> | <b>0.68x</b> | <b>53.8%</b> | <b>42.7%</b>  | <b>84.5%</b> |

### 7.3 Ablation Studies: Target Sparsity and PI Gains

To verify the stability and performance impact of the controller parameters, we performed a thorough hyperparameter ablation over  $K_p$ ,  $K_i$ , and the target sparsity limits. The results are summarized in Table 2.

**Table 2.** Hyperparameter Ablation of PI-Controller Settings

| Target Sparsity | $K_p$       | $K_i$        | Final Average $k_{dyn}$ | Avg. FLOPs   | DeepPlanning | AIME 2025    |
|-----------------|-------------|--------------|-------------------------|--------------|--------------|--------------|
| 0.50            | 0.05        | 0.005        | 3.4                     | 0.82x        | 51.9%        | 82.1%        |
| 0.60            | 0.10        | 0.010        | 2.8                     | 0.73x        | 52.8%        | 83.4%        |
| <b>0.70</b>     | <b>0.15</b> | <b>0.015</b> | <b>2.1</b>              | <b>0.68x</b> | <b>53.8%</b> | <b>84.5%</b> |
| 0.80            | 0.20        | 0.020        | 1.4                     | 0.52x        | 48.2%        | 78.1%        |

Our ablation findings suggest that pushing the Target Sparsity to 0.80 restricts the active computation budget too severely (reducing average active experts  $k_{dyn}$  to 1.4), causing a noticeable degradation in reasoning-dense tasks such as AIME 2025. Conversely, setting Target Sparsity to 0.50 activates too many experts on simple tokens, matching baseline performance but losing the desired FLOP savings. The optimal parameter configuration is reached at a Target Sparsity of 0.70, yielding high reasoning accuracy with a 32% reduction in active FLOPs.

### 7.4 Case Study: The Coupon Stacking Scenario

In the DeepPlanning shopping task, an agent must minimize costs across multiple stores with complex coupon logic. We observed that  $H(x)$  averages 0.12 during routine item listing (Fast Mode) but spikes to 0.94 when encountering the overlapping coupon constraints. EGTB successfully allocates the full thinking budget (12 experts + internal CoT) during these critical spikes. In contrast, fixed Top-k models often fail to capture the combinatorial nuances, resulting in a 15% lower completion rate for Level 3 shopping tasks [11, 15].

During our execution logs of the Coupon Stacking task, the transition from routine text parser blocks to logic-heavy matching rules showed an instantaneous transition of routing entropy. For filler tokens (e.g., "then", "with", "and"), the router concentrated 98% of its probability mass on Expert 3 (syntax block), giving an entropy of  $H(x) \approx 0.08$ . However, when encountering conditional numerical arrays representing coupon thresholds, the distribution scattered across Experts 12, 14, and 22, causing  $H(x)$  to rise to 2.41, which immediately triggered the maximum thinking budget.

## 8 Discussion: Impact on Hardware and Scale

One of the primary failure modes of agentic LLMs in 1M-token contexts is "Reasoning Exhaustion," where the model's consistency degrades over long trajectories [12, 16]. EGTB addresses this by treating compute as a strategic resource.

By staying in “Fast Mode” for 74% of tokens (mostly linguistic fillers), the system preserves its computational stamina for the global optimization phases of a plan.

### 8.1 Hardware-Centric Performance and Roofline Analysis

Modern hardware platforms such as the NVIDIA H100 are heavily bound by memory bandwidth during the autoregressive generation phase. In standard MoE layers, loading the weights of all Top- $k$  experts from High Bandwidth Memory (HBM) to local SRAM cache for single-token inferences introduces massive latency penalties.

By applying EGTB, we alter the hardware execution characteristics in two key ways:

1. **Kernel Coalescing:** Since 74% of tokens bypass heavy expert blocks via the Fast Mode/Null-Expert routes, we dramatically reduce SRAM-to-HBM load requests, allowing the hardware to remain execution-bound rather than memory-bound for long sequences.
2. **Dynamic Batching:** We group tokens of similar entropy classes together in the pipeline, executing low-entropy tokens on shared base modules and restricting expensive multi-GPU all-to-all expert communication to high-entropy tokens only.

### 8.2 System-Level Optimization

EGTB is optimized for inference backends like vLLM. By leveraging the *LM-Cache* layer, EGTB stores entropy profiles for common prompts, allowing the pre-allocation of budgets during the prefill stage, thereby removing the overhead of real-time entropy calculation in subsequent turns [13]. This cache utilization enables multi-agent pipelines to share structural routing profiles, minimizing latency jitter on distributed clusters.

## 9 Conclusion

Entropy-Gated Thinking Budgets offer a principled solution to the Uniform Sparsity Paradox. By coupling information-theoretic signals with classic control theory, we enable agents to navigate the intelligence-efficiency trade-off autonomously. Our findings indicate that dynamic scaling is not only faster but leads to superior reasoning outcomes by focusing compute on high-impact decision points. Future work will investigate the application of EGTB to multimodal architectures for embodied reasoning in robotics.

## References

1. Step 3.5 Flash Technical Report, arXiv:2602.10604 (2026).

2. Qwen3 Technical Report: Integration of Thinking and Non-Thinking Modes (2025).
3. Fedus et al., Switch Transformers: Scaling to Trillion Parameter Models, JMLR (2021).
4. DeepSeek-V3 Technical Report: Fine-Grained Sparse Attention (2024).
5. LongCat-Flash: Zero-Computation Experts, arXiv:2509.01322 (2025).
6. OpenAI o1-series: Inference-Time Optimization, Technical Report (2024).
7. Graves, A., Adaptive Computation Time for RNNs, arXiv:1603.08983 (2016).
8. Optimal Sparsity of Mixture-of-Experts Transformers, arXiv:2508.18672 (2026).
9. DTop-p MoE: Sparsity-Controllable Dynamic Routing, arXiv:2512.13996 (2025).
10. Li et al., Entropy-Gated Branching for Reasoning, arXiv:2503.21961 (2026).
11. DeepPlanning: Benchmarking Long-Horizon Planning, Qwen Team (2026).
12. AgencyBench: Frontiers of Autonomous Agents, arXiv:2601.11044 (2026).
13. LMCACHE: Efficient KV Cache Layer for LLM Inference (2026).
14. Adaptive-K routing: Compute savings on MoE models, LocalLLaMA (2026).
15. Lepikhin, D., et al. GShard: Scaling Giant Models with Conditional Computation and Automatic Sharding, arXiv:2006.16668 (2020).
16. X. Zhang, et al., UltraHorizon: Benchmarking LLM-Agent Capabilities in Ultra-Long-Horizon Discovery Tasks, arXiv:2509.21766 (2025).
17. J. Wang, et al., DARE: Difficulty-Aware Dynamic Routing for Mixture of Experts, (2026).